

## Structure Execution into Memory Traces

### Execution History

- Observation**  
"Read error..."
- Edit**  
"Modify code..."
- Test**  
"Run tests"
- Tool Output**  
"Test passed."
- Decision**  
"Refactor..."
- Milestone**  
semantic boundary



### Memory Trace

- Body**  
execution evidence
- Repo State**  
snapshot / commit
- Entities**  
files / symbols / tools
- Memory Anchor**  
address binding



### Memory Episode

- T1 → T2 → T3 → T4 ...
- Goal / Task**  
e.g. fix parser module
- Traces**  
ordered memory traces
- Summary**  
key progress, decisions, outcome
- Metadata**  
time range, related files, current status



### Trace Store

Immutable  
Never Overwritten

- Memory traces
- Memory Episode
- Blob
- Provenance
- Memory Anchor

## Addressable Working Context

### Working Context

Recent  
Context

Active  
Task

Full  
Traces

reaching limit

replace

still full

### Memory Offloading



Persist full content  
externally



### Memory Index

| AnchorID | Type  | Section | Version | Metadata  |
|----------|-------|---------|---------|-----------|
| m_001    | trace | p_1-10  | v3      | parser.py |
| m_002    | trace | p_20-28 | v1      | tests/    |
| m_003    | code  | p_5-12  | v2      | utils.py  |
| ...      | ...   | ...     | ...     | ...       |

Resolve to  
trace

lookup

relational routing  
Memory trace Graph  
(MTG)

Execution Relations



updated with  
execution events

Structural hints  
Repository State Graph  
(RSG)

Repository Structure



updated with  
repository state changes

Shared  
Repository Entity  
File  
Symbol  
Test  
...

Epoch limit reached  
· Logical Fallback Exhausted  
· Context Cannot Fit  
· Thread / Provider fails

Create Checkpoint  
· Current Task State  
· Relevant Memory Traces  
· Relevant Code Files  
· Execution Frontier

New Epoch  
· Task State  
· Memory Traces  
· Code Files  
· Execution Frontier

Continue Execution  
Seamless long-horizon  
progression!

## Minimal Recall with Validation

### Trigger Recall

Context  
Refresh

Recall Intent

Memory  
Anchor / Query

Guided by  
MTG and RSG

### Memory Index Translation

①Memory Trace

# Trace: T17

- 2025-01-12 10:24
- Fix failing test
- sklearn/svm/
- Read files
- Edit code
- Run tests
- Error logs
- Discussion
- ...

②Section

Section: Implementation

- Modify decision\_function in base.py
- Edit hunks (diff)
- Run related tests
- ...

Metadata

type: code\_modification  
files: sklearn/svm/base.py

③Evidence Range

Range: R3(base.py)

④Minimal Evidence

Lines: 771-773

```
771 + # fixed: handle empty input
772 + if X.size == 0:
773 +     return np.array([])
```

```
768 def decision_function(self, X):
769 - if self.probability:
770 -     return self._predict_proba_lr(X)
771 + # fixed: handle empty input
772 + if X.size == 0:
773 +     return np.array([])
774     return self._decision_function(X)
```

Validated against  
Repository State and Task State

Validated  
Reload

Inject to  
Working Context